



**Universidad
Europea** MADRID

PROYECTO INTEGRADOR

1DAW

HOOLIGANS DAW



ENZO RUIZ RAÉZ

GUILLERMO JIMENEZ MEDINA

LUCAS LOBERA MUÑOZ

Índice

Resumen	2
1. Introducción	3
2. Objetivos	3
3. Tecnologías utilizadas	3
4. Desarrollo e implementación	3
5. Metodología	3
6. Resultados y conclusiones	4
7. Trabajos futuros	4
Anexos	5
Anexo I – Listado de requisitos de la aplicación	5
Anexo II – Guía de uso de la aplicación	5
Anexo III	5

Resumen

Este Proyecto Integrador consiste en el desarrollo de una aplicación de gestión de citas para el taller de Edna Moda, orientada a facilitar la organización de clientes, trajes, empleados y citas dentro del negocio. El sistema permite realizar operaciones CRUD (crear, consultar, modificar y eliminar) sobre los diferentes elementos, utilizando una base de datos relacional diseñada específicamente para este propósito.

Durante el desarrollo se han aplicado los conocimientos adquiridos en los módulos de Programación, Bases de Datos y Entornos de Desarrollo, implementando una arquitectura basada en el patrón MVC que permite separar la lógica, los datos y la interfaz de usuario. Además, se ha trabajado con herramientas como Java, MySQL, Eclipse y GitHub, facilitando tanto la implementación técnica como el trabajo colaborativo del equipo.

El proyecto se ha llevado a cabo siguiendo la metodología ágil SCRUM, dividiendo el trabajo en distintos sprints que han permitido una evolución progresiva del sistema. Esto ha favorecido la organización del equipo, la adaptación a cambios y la mejora continua del producto.

Como resultado, se ha desarrollado una aplicación funcional que cumple con los requisitos planteados, incluyendo control de acceso por roles (aprendiz, oficial y maestro) y gestión completa de la información del taller.

1. Introducción

En este documento presentamos el desarrollo de nuestro Proyecto Integrador del primer curso del ciclo de Desarrollo de Aplicaciones Web (DAW). Nuestro grupo, Hooligans Daw, ha trabajado en el diseño y creación de una aplicación que permite gestionar información sobre clientes, trajes, empleados y citas dentro de un taller.

La idea principal ha sido crear una herramienta que facilite la organización del trabajo, ya que en lugar de usar métodos manuales, con esta aplicación se pueden gestionar los datos de forma más rápida y sencilla. Para ello, hemos creado bases de datos que permiten almacenar y manejar toda esta información, además de implementar las operaciones básicas como añadir, consultar, modificar y eliminar datos.

Durante el desarrollo hemos utilizado los conocimientos aprendidos en asignaturas como Programación, Bases de Datos y Entornos de Desarrollo. También hemos trabajado con una base de datos relacional y hemos estructurado la aplicación siguiendo el modelo MVC para organizar mejor el código.

Además, el trabajo se ha realizado en grupo utilizando herramientas como GitHub, lo que nos ha permitido trabajar de forma colaborativa, organizar mejor el proyecto y llevar un control de los cambios realizados.

2. Objetivos

Los objetivos principales que tenemos con este proyecto son:

- Desarrollar una aplicación capaz de gestionar clientes, trajes, empleados y citas.
- Diseñar y crear una base de datos relacional para almacenar toda la información.
- Implementar operaciones básicas de gestión de datos (crear, consultar, modificar y eliminar).
- Aplicar los conocimientos adquiridos en Programación, Bases de Datos y Entornos de Desarrollo.
- Organizar la aplicación siguiendo el modelo MVC.
- Trabajar en equipo utilizando herramientas como GitHub.
- Planificar y desarrollar el proyecto utilizando una metodología ágil.

3. Tecnologías utilizadas

Para el desarrollo de este proyecto hemos utilizado las siguientes tecnologías y herramientas:

- Java como lenguaje de programación principal.
- MySQL para la creación y gestión de la base de datos.
- Oracle APEX para trabajar y probar la base de datos.
- Eclipse como entorno de desarrollo (IDE).
- GitHub para el control de versiones y el trabajo en equipo.
- Trello para la organización de tareas.
- Herramientas de ofimática como Microsoft Word para la documentación.
- Design.com para la creación del logo de Edna Moda

4. Desarrollo e implementación

En este apartado explicamos cómo hemos desarrollado nuestra aplicación, desde el diseño inicial hasta su funcionamiento final.

En primer lugar, hemos elaborado los distintos diagramas que nos han servido como base para construir el proyecto, como el modelo entidad-relación, el modelo relacional, el diagrama de casos de uso y el diagrama de clases. Estos diagramas nos han permitido organizar la información, definir las relaciones entre los datos y estructurar correctamente el código.

A partir de este diseño, hemos desarrollado la aplicación en Java siguiendo el patrón MVC, lo que nos ha ayudado a separar la lógica del programa, la gestión de datos y la interfaz de usuario. De esta forma, hemos conseguido una estructura más clara y fácil de mantener.

También hemos conectado la aplicación con una base de datos en MySQL, donde almacenamos toda la información relacionada con clientes, trajes, empleados, talleres y citas. Esto nos permite realizar operaciones como crear, consultar, modificar y eliminar datos de forma eficiente.

Además, hemos diseñado un logo representativo del proyecto, buscando una imagen sencilla pero acorde con la temática del taller de moda.

Por último, hemos realizado diferentes pruebas para comprobar que la aplicación funciona correctamente, verificando tanto la gestión de datos como el control de accesos según el tipo de usuario.

En los siguientes apartados se detallan todos estos elementos para mostrar el resultado final del proyecto.

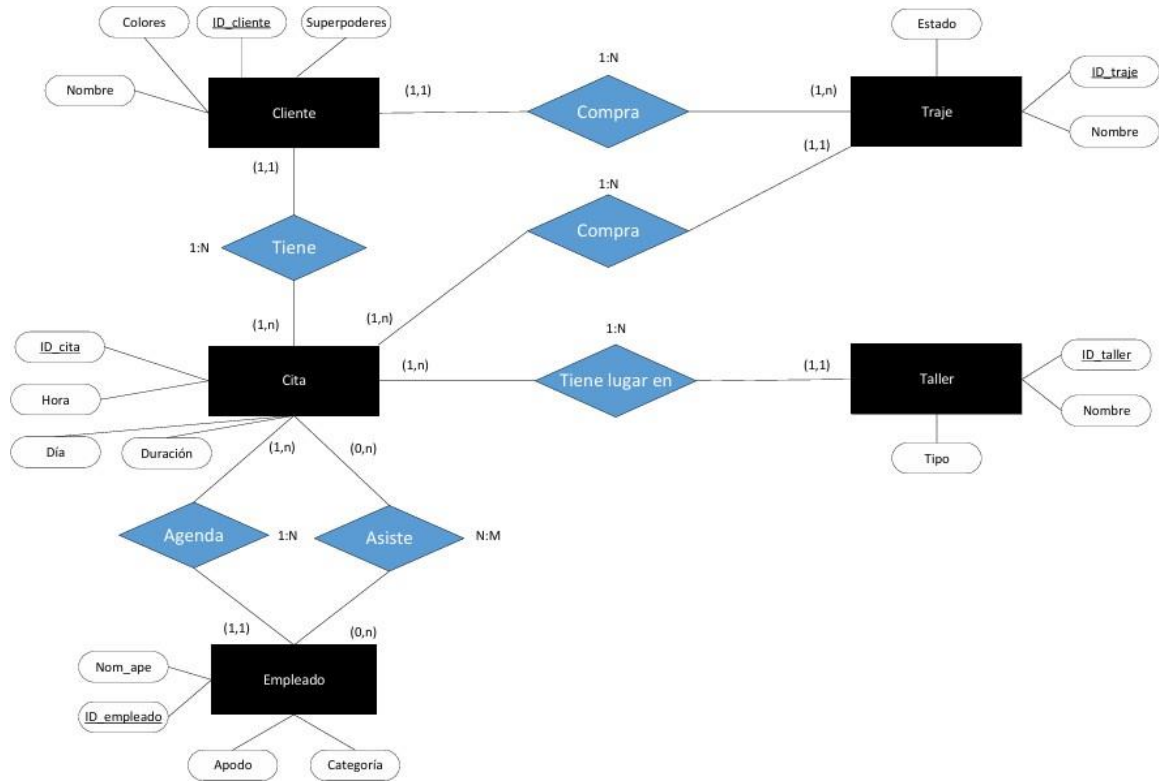
LOGO EDNA MODA



4.1.1 Modelo Entidad Relación

Para organizar la información, primero hemos diseñado un modelo conceptual donde identificamos las entidades y cómo se conectan. Basándonos en nuestro diagrama, las entidades principales son:

- **Cliente:** Representa a las personas que usan el servicio. Guardamos su nombre, colores preferidos y superpoderes.
- **Cita:** Es el eje central del taller. Registra el día, la hora y la duración de cada visita.
- **Traje:** Los artículos que se gestionan, con su nombre y estado actual.
- **Empleado:** El personal del taller, clasificado por su categoría y apodo.
- **Taller:** El lugar físico donde ocurren las citas.



4.1.2 Diseño Modelo Relacional y Normalización

En esta fase, hemos convertido nuestro diseño conceptual en tablas reales y las hemos optimizado para que el sistema funcione sin errores.

Modelo Relacional

Hemos definido la estructura de la base de datos mediante las siguientes tablas y sus conexiones:

- **Cliente y Traje:** Registramos los datos del cliente, como su nombre y superpoder. Cada traje se asocia a un cliente específico mediante su ID.
- **Taller y Empleado:** Guardamos la información de los lugares de trabajo y del personal, clasificado por categorías.
- **Cita:** Es la tabla central que conecta un día y hora con un taller, un responsable, un traje y el cliente correspondiente.
- **Asiste:** Una tabla especial para registrar qué empleados participan en cada cita.

Normalización

Para que la base de datos sea profesional y eficiente, hemos aplicado un proceso de limpieza en tres pasos:

1. **Primera Forma Normal (1FN):** Nos aseguramos de que cada celda de nuestras tablas contenga un solo dato.
2. **Segunda Forma Normal (2FN):** Hemos verificado que todos los datos de una tabla dependen directamente de su identificador único o ID.
3. **Tercera Forma Normal (3FN):** Hemos eliminado datos que sobraban. Un cambio importante fue en la tabla Cita, donde quitamos el ID del cliente, esto se hizo porque esa información ya está guardada en la tabla del traje, evitando así repetir datos innecesariamente.

 **Relacional y normalización.pdf**

4.1.3 Diagrama de casos de uso

Este diagrama explica qué puede hacer cada perfil dentro del programa:

- **Maestro:** Tiene el control total. Puede gestionar clientes añadir, borrar o modificar y organizar todas las citas del taller.
- **Oficial:** Puede ver todas las citas, crear nuevas y modificar las suyas propias. También tiene permiso para asignar aprendices a los trabajos.
- **Aprendiz:** Es el perfil con más restricciones; solo puede entrar para consultar las citas que tiene asignadas.
- **Login:** Por seguridad, cualquier usuario debe identificarse con sus credenciales antes de acceder a estas funciones.

 **PI.pdf**

4.1.4 Diagrama de clases

En este diagrama de clases se explica cómo está organizado el código en Java usando el modelo MVC:

- **Vista:** Pantallas de Login y Menú para el usuario.
- **Modelo:** Clases que guardan los datos de Clientes, Trajes, Citas y Empleados.
- **Controlador:** Clases que conectan el programa con la base de datos de MySQL.

 **DiagramaDeClasesPI.pdf**

4.2 Funcionamiento de la aplicación

La aplicación desarrollada permite gestionar de forma completa la actividad del taller de Edna Moda mediante una interfaz sencilla e intuitiva. El sistema comienza con una pantalla de inicio de sesión (login), donde los empleados introducen sus credenciales para acceder. Una vez dentro, el usuario accede a un menú principal desde el que puede gestionar citas, clientes y talleres, dependiendo de su nivel de permisos. Existen tres tipos de usuarios:

- Aprendiz: solo puede consultar las citas asignadas.
- Oficial: puede ver todas las citas, crear nuevas y modificar las suyas.
- Maestro: tiene control total sobre el sistema, pudiendo gestionar clientes, talleres y citas.

El sistema permite crear citas indicando cliente, traje, empleado, taller, fecha y duración. Además, se pueden modificar o eliminar registros existentes, garantizando una gestión completa de la información.

4.3 Relación con la base de datos

La aplicación está conectada a una base de datos relacional desarrollada en MySQL. Esta base de datos contiene las tablas necesarias para almacenar la información del sistema: clientes, trajes, empleados, talleres y citas.

La conexión se realiza desde el código Java mediante clases específicas del modelo y controlador, que se encargan de ejecutar consultas SQL para insertar, actualizar, eliminar y recuperar datos. Esto permite que la aplicación funcione de manera dinámica, reflejando en todo momento los cambios realizados por el usuario.

El diseño de la base de datos sigue las normas de normalización hasta la tercera forma normal (3FN), lo que garantiza la integridad de los datos y evita redundancias innecesarias.

4.4 Aplicación del modelo MVC

La aplicación sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador), lo que permite organizar el código de manera clara y eficiente.

- **Modelo:** contiene las clases que representan los datos (clientes, citas, empleados, etc.) y gestionan el acceso a la base de datos.
- **Vista:** incluye las interfaces gráficas con las que interactúa el usuario, como las pantallas de login y menús.
- **Controlador:** actúa como intermediario entre la vista y el modelo, gestionando las acciones del usuario y coordinando las operaciones.

Esta separación facilita el mantenimiento del código, mejora la organización del proyecto y permite futuras ampliaciones de forma más sencilla.

4.5 Diseño del logo

El logo de la aplicación ha sido diseñado utilizando herramientas online, con el objetivo de representar la identidad del proyecto de forma visual.

Se ha optado por un diseño sencillo y elegante, acorde con la temática del taller de moda.

Los elementos visuales y tipográficos buscan transmitir profesionalidad y creatividad, aspectos clave del entorno en el que se desarrolla la aplicación.

La elección de colores y formas se ha realizado pensando en una imagen clara y fácilmente reconocible, que pueda asociarse al sistema desarrollado.

4.6 Pruebas realizadas

Durante el desarrollo del proyecto se han realizado diversas pruebas para comprobar el correcto funcionamiento de la aplicación.

Se han llevado a cabo pruebas manuales para verificar:

- El inicio de sesión con distintos tipos de usuario.
- La creación, modificación y eliminación de citas.
- La gestión de clientes y talleres.
- El correcto funcionamiento de los permisos según el rol del usuario.

Además, se han comprobado posibles errores en la conexión con la base de datos y en la validación de datos introducidos por el usuario.

Estas pruebas han permitido detectar y corregir errores, mejorando la estabilidad y el rendimiento de la aplicación.

5. Metodología

Hemos utilizado la metodología ágil SCRUM para el desarrollo de este proyecto, ya que nos ha permitido trabajar de forma flexible, organizada y colaborativa. Esta metodología resulta especialmente adecuada en entornos donde los requisitos pueden cambiar a lo largo del tiempo, ya que facilita la adaptación continua y la mejora progresiva del producto.

La elección de SCRUM responde a la necesidad de gestionar el proyecto de manera iterativa, dividiendo el trabajo en ciclos cortos (sprints) que permiten evaluar resultados de forma frecuente y realizar ajustes cuando es necesario. Además, fomenta la autoorganización del equipo y una comunicación constante entre sus miembros, lo que ha contribuido a mejorar la coordinación y la eficiencia en el desarrollo.

Asimismo, SCRUM nos ha ayudado a afrontar los imprevistos y cambios en los requisitos del cliente como una oportunidad de mejora, en lugar de un obstáculo, permitiéndonos ofrecer una solución más ajustada a las necesidades reales del proyecto.

Durante el desarrollo del proyecto, el equipo ha trabajado en distintos sprints organizados mediante la herramienta Trello, donde se han asignado tareas y realizado el seguimiento del progreso.

Cada miembro del equipo ha contribuido en diferentes partes del proyecto, como el diseño de la base de datos, la programación de la aplicación o la elaboración de la documentación. Además, se han realizado reuniones periódicas para coordinar el trabajo y resolver problemas.

El uso de GitHub ha permitido gestionar versiones del proyecto y trabajar de forma colaborativa, facilitando la integración del código desarrollado por cada miembro.

6. Resultados y conclusiones

El desarrollo de este proyecto ha permitido crear una aplicación funcional que cumple con los requisitos planteados inicialmente. Se ha conseguido implementar un sistema completo de gestión de citas, clientes, trajes y empleados, utilizando una base de datos relacional y una arquitectura basada en el modelo MVC.

El resultado final es una aplicación capaz de gestionar la información de forma eficiente, permitiendo realizar operaciones CRUD y controlar el acceso según el tipo de usuario. Además, se ha logrado integrar correctamente la base de datos con la aplicación Java, asegurando la persistencia de los datos.

A nivel de equipo, el uso de herramientas como GitHub y Trello ha facilitado la organización del trabajo y la colaboración entre los miembros. La aplicación de la metodología SCRUM ha permitido dividir el proyecto en fases, facilitando su desarrollo progresivo.

Como conclusión, este proyecto ha servido para aplicar de manera práctica los conocimientos adquiridos durante el curso, así como para mejorar habilidades técnicas y de trabajo en equipo.

7. Trabajos futuros

En futuras versiones del proyecto se podrían implementar diversas mejoras para ampliar las funcionalidades de la aplicación.

Entre ellas, se podría desarrollar una interfaz gráfica más avanzada y atractiva para mejorar la experiencia de usuario. También sería interesante añadir nuevas funcionalidades, como la gestión de horarios automáticos o la notificación de citas.

Otra posible mejora sería reforzar la seguridad del sistema, implementando sistemas de encriptación de contraseñas y control de acceso más avanzado.

Además, se podría adaptar la aplicación a un entorno web, permitiendo su acceso desde cualquier dispositivo con conexión a internet.

Estas mejoras permitirían evolucionar el proyecto hacia una aplicación más completa y profesional.

ANEXOS

Anexo I – Listado de requisitos de la aplicación

La aplicación debe gestionar información almacenada en bases de datos relacionales.

- Se debe permitir la creación, borrado y cambio de citas y acceso a talleres y clientes.
- Debe contar con una pantalla de log in.
- La aplicación debe proporcionar diferentes permisos dependiendo de la categoría del empleado que inicie sesión.
- Debe seguir el modelo MVC (Modelo, Vista, Controlador).

Anexo II – Guía de uso de la aplicación

Incluirse en este anexo capturas de la aplicación, a modo de manual de usuario, incluyendo tanto el acceso de usuario, como de admin.

Anexo III – Recursos adicionales

En este apartado se incluyen enlaces y recursos utilizados durante el desarrollo del proyecto:

- Repositorio GitHub del proyecto:
<https://github.com/enzo2212ruiz/Hooligans-DAW.git>
- Tablero de organización en Trello:
<https://trello.com/b/xAOIWaPt/pi-hooligans-daw>
- Diagramas y documentación adicional generada durante el desarrollo.